CLIENTES FTP

Descargar PDF

- CLIENTES FTP
  - INTRODUCCIÓN
  - CLIENTES TEXTO
    - FTP
    - SFTP
    - LFTP
  - CLIENTES GRÁFICOS
    - EXPLORADORES DE ARCHIVOS
    - OTROS CLIENTES

INTRODUCCIÓN

Un **cliente FTP** es una aplicación que permite conectarse a un servidor FTP para **subir, descargar y administrar archivos**. En general los clientes FTP ejecutan comandos o acciones que convierten en comandos RAW del protocolo FTP. Por ejemplo, un cliente puede ejecutar el comando `user usuario contraseña` que se convertirá en los comandos RAW `user usuario` y `pass contraseña` enviados a través del canal de control (puerto 21) del protocolo FTP. La mayor parte de los clientes FTP actuales **soportan** los protocolos : **FTP**, **FTPS** (FTP over TLS/SSL) y **SFTP** (FTP over SSH).

Tenemos fundamentalmente dos tipos de clientes:

- **Cliente Texto**
  Los clientes en modo texto funcionan mediante una terminal o consola y permiten instroducir comandos simples para ejecutar acciones en el servidor FTP. Algunos ejemplos son: `ftp` , `lftp` , `sftp`
- **Cliente Gráfico**
  Los clientes gráficos añaden una interfaz visual y una gestión de sesiones más intuitiva. Por ejemplo, los exploradores de archivo de Windows y Linux pueden actuar con algunas limitaciones como clientes FTP sencillos. Algunos ejemplos de clientes más complejos son: `WinSCP` , `FileZilla` , `CuteFTP` .

CLIENTES TEXTO

FTP

El cliente `ftp` es un cliente estándar presente por defecto en muchas distribuciones de Linux y en Windows. Características:

- Conexión simple a servidores FTP.
- Navegación de directorios y transferencia de archivos.
- Ejecución de comandos básicos de FTP en terminal.

El **cliente ftp de linux** (man) sigue la siguiente sintáxis

```
ftp [-pinegvd] [host]

-p Usar el modo pasivo. Por defecto, modo activo para datos.
-i Modo no interactivo.
-n Deshabilitar auto-login al realizar conexión.
-e Deshabilitar historial y edición de linea de comandos.
-g Deshabilitar caracteres comodin (*,?,[]) en nombres de archivo.
-m Deshabilitar usar la misma interfaz de red para canal de control y datos.
-v Modo verbose, es decir, detallado.
-d Modo debug
```

Conexión FTP

Conectar con el servidor ftp.ejemplo.com usando el modo no interactivo, pasivo, debug y sin autologin.

```
ftp -ipdn ftp.ejemplo.com
```

Una vez ejecutado el cliente, entramos en el modo consola del cliente FTP. En este modo podemos ejecutar los siguientes **comandos**:

Comando	Descripción
Sesión	
`open <host> <port>`	Conecta al servidor FTP. Si no se indica , usa el 21.
`user <user-name> <password> <account>`	Envía usuario y contraseña. Si el servidor requiere un account, el cliente solicitará o usará el valor dado.
`account <passwd>`	Envía una segunda contraseña (cuenta adicional) si el servidor la exige tras iniciar sesión.
`status`	Muestra el estado actual de la sesión, incluyendo tipo de transferencia, modo y conectividad.
`system`	Muestra información del sistema remoto (resultado de FTP SYST).
`idle <seconds>`	Obtiene o fija el tiempo de inactividad permitido por el servidor para la sesión actual (timeout).
`reset`	Vacía la cola de respuestas del servidor y resincroniza la sesión cuando hay desajustes.
`close` `disconnect`	Cierra la conexión con el servidor pero mantiene el cliente FTP abierto en el prompt.

`bye`
`quit`
`exit`
	Cierra la conexión y sale completamente del cliente FTP.

Transferencia	
`passive`	Activa/desactiva modo pasivo.
`ascii`	Cambia el modo de transferencia de datos a ASCII.
`binary` `image`	Cambia el modo de transferencia de datos a la BINARIO.
`type <type-name>`	Muestra o establece el tipo de transferencia de datos. - ASCII - BINARY - IMAGE (similar BINARY)
`mode <mode-name>`	Configura el modo de transferencia de datos. -STREAM flujo de bytes (por defecto) -BLOCK bloques con cabecera y control -COMPRESSED comprimidos.
`hash`	Imprime un `#` por cada bloque transferido (1024 bytes).
`restart <bytecount>`	Continúa transferencia de datos desde un punto específico.

Comando	Descripción
Ficheros	
`append <lfile> <rfile>`	Añade un archivo local a uno remoto (lo crea si no existe).
`put <lfile> <rfile>` `send`	Sube un archivo local a uno remoto.
`get <rfile> <lfile>` `recv`	Descarga un archivo remoto a uno local.
`newer <rfile> <lfile>`	Descarga remoto sólo si es más reciente que en el local.
`delete <rfile>`	Elimina un archivo remoto.
`rename <rold> <rnew>`	Renombra un archivo remoto.
`chmod <mode> <rfile>`	Cambia permisos en archivo remoto. Permisos tipo linux (rwx), pero depende del servidor.
`modtime <rfile>`	Muestra fecha de modificación de un archivo remoto.
`size <rfile>`	Muestra el tamaño del archivo remoto.
Directorios	
`pwd`	Muestra el directorio remoto actual.
`mkdir <rdir>`	Crea un directorio remoto
`rmdir <rdir>`	Elimina directorio remoto.
`dir <rdir>`	Lista el directorio remoto formato largo
`ls  <rdir>`	Lista el directorio remoto formato corto
`nlist <rdir>`	Lista directrio remoto solo nombres.
`cd <rdir>`	Cambia el directorio remoto.
`cdup`	Cambia al directorio padre remoto.
Multiple	
`prompt`	Activa/desactiva consultas interactivas en operaciones múltiples.
`glob`	Activa/desactiva expansión de comodines (*,?,[]) en nombres de archivos.
`nmap`	Establece plantillas para renombrado automático.
`ntrans`	Define tabla de traducción para nombres.
`mput <rfile>`	Sube múltiples archivos locales al servidor
`mget <rfile>`	Descarga múltiples archivos remotos.
`mdelete <rfile>`	Elimina múltiples archivos remotos.
`mdir <rfile>`	Lista múltiples archivos remotos

Comando	Descripción
`mls <rfile>`	Lista múltiples archivos remotos
Local	
`!<cmd>`	Ejecuta un comando del shell local o abre una shell interactiva.
`lcd <ldir>`	Cambia directorio local en el cliente.
Otros	
`help <cmd>`	Muestra ayuda del cliente FTP o de un comando concreto
`verbose`	Activa salida detallada
`debug`	Activa/desactiva salida detallada de mensajes FTP.
`bell`	Activa/desactiva un sonido al finalizar cada comando.
`literal <args>` `quote <args>`	Envía comandos FTP sin interpretar.
`$macro <args>`	Ejecuta una macro definida con `macdef`
`macdef name`	Define una macro. Las lineas siguientes son guardadas en la macro hasta que haya una linea en blanco.
`proxy <cmd>`	Ejecuta un comando usando una conexión de control secundaria.

Los comandos FTP de tratamiento de ficheros o directorios no son recursivos, por ejemplo, mget no descarga recursivamente directorios.

Conexión FTP cliente LINUX

Usar el cliente ftp en modo no interactivo, debug y sin autologin. Ejecutar el cliente con comandos FTP contenidos en un archivo de texto que permitan conectarse con ftp.ejemplo.com para el usuario alumno y password p4bl1c0. Las transferencias se realizarán en modo pasivo. Listar el contenido del directorio /PUBLICO/APUNTE y descargar su contenido en directorio local /TEMP/DESCARGAS.

```
# contenido de ftp_in.txt
open ftp.ejemplo.com
user alumno p4bl1c0
passive
debug
lcd /TEMP/DESCARGAS
cd /PUBLICO/APUNTES
ls
mget *
bye
```

```
# Ejecutar cliente ftp y redirigir la salida a un archivo.
ftp -invd < ftp_in.txt >ftp_out.txt 2>&1
```

El **cliente FTP de Windows** (help) sigue la siguiente sintáxis

```
ftp [-v] [-d] [-i] [-n] [-g] [-s:archivo] [-a] [-A] [host]

-v  Desactiva mostrar respuestas del servidor.
-n  Desactiva inicio de sesión automático
-i  Desactiva la intervención interactiva.
-d  Activa la depuración.
-g  Desactiva comodines en nombres de archivos.
-s:archivo Especifica archivo con comandos de FTP
-a Usa cualquier interfaz local para datos.
-A Inicio de sesión anónimo.
host IP o nombre del servidor
```

Una vez ejecutado el cliente, entramos en el modo consola del cliente FTP. Los **comandos** del cliente FTP de Windows son similares a los de Linux aunque tienen menos opciones y fundamentalmente **NO** funciona en **modo pasivo** lo que le hace inadecuado para trabajar en redes con NAT.

Lista de comandos FTP Windows.

```
!      delete     literal   prompt    send
?      debug      ls        put       status
append dir        mdelete   pwd       trace
ascii  disconnect mdir      quit      type
bell   get        mget      quote     user
binary glob       mkdir     recv      verbose
bye    hash       mls       remotehelp
cd     help       mput      rename
close  lcd        open      rmdir
```

> Los comandos mget and mput aceptan s/n/c como respuesta automática para sí/no/cancelar.

Conexión FTP cliente Windows

Sobre el mismo supuesto que el ejemplo anterior del cliente Linux.

```
# contenido de ftp_in.txt. No activamos passive.
open ftp.ejemplo.com
user alumno p4bl1c0
debug
lcd /TEMP/DESCARGAS
cd /PUBLICO/APUNTES
ls
mget *
bye
```

```
# ejecutar cliente ftp y redirigir la salida a un archivo.
ftp -i -n -v -d -s:ftp_in.txt >ftp_out.txt 2>&1
```

Aqui tienes una relación simple entre los comandos de un cliente FTP y los **comandos raw** del Protocolo FTP.

cliente FTP	raw FTP
`ls` / `dir`	`LIST`
`cd`	`CWD`
`pwd`	`PWD`
`get`	`RETR`
`mget`	`RETR` múltiples
`put`	`STOR`
`mput`	`STOR` múltiples
`mkdir`	`MKD`
`rmdir`	`RMD`
`delete`	`DELE`
`quit`	`QUIT`

SFTP

El cliente `sftp` (man) es un subsistema seguro de SSH que permite transferir archivos, gestionar directorios y ejecutar operaciones remotas de forma cifrada. Podemos utilizarlo tanto en sistemas Linux como Windows. Entre sus características tenemos:

- Incluido en el paquete OPENSSH cliente.
- Uso del puerto 22 de SSH para la transmisión de la información de control y de datos. No necesita modo activo y/o pasivo.
- Transmisión cifrada de la información.
- Uso de claves privadas y/o publicas.

El **cliente SFTP** sigue la siguiente sintáxis

```
sftp [options] user@host
Entre las opciones más comunes tenemos:

-a          Continuar transferencias interrumpidas temporalmente.
-b          Fichero con comandos sftp para ejecutar en la conexión.
-C          Habilitar compresión
-c <cipher> Usar <cipher> para cifrar datos
-i <idfile> Usar <idfile> como clave privada para logging remoto.
-o <option> Usar <option> como valor opcional para SSH.
            Las opciones sobreescriben las del fichero ssh_config.
-P <port>   Usar <port> como puerto de conexión en lugar del 22.
-r          Modo recursivo por defecto para transferencias de datos.
```

En un shell de `sftp` podemos ejecutar **comandos similares** a los de un cliente FTP:

Comando	Descripción
Sesion	
`version`	Muestra la versión de sftp

`bye`
`quit`
`exit`
	Cierra la conexión y sale del cliente SFTP.

Ficheros y Directorios	
`put <options> <lpath> <rpath>`	Sube un archivo o directorio local a uno remoto.Entre las opciones tenemos: `-a` Continuar transferencias parcialmente interrumpidas `-f` Invocar funcion fsync una vez la transferencia ha terminado. `-P` Copiar permisos y propiedades del archivo `-R` modo recursivo
`get <options> <rpath> <lpath>`	Descarga un archivo o directorio remoto a uno local. Entre las opciones tenemos: `-a` Continuar transferencias parcialmente interrumpidas `-f` Invocar funcion fsync una vez la transferencia ha terminado. `-P` Copiar permisos y propiedades del archivo `-R` modo recursivo
`ls <rpath>`	Lista el archivo o directorio remoto.
`cp <options> <roldpath> <rnewpath>`	Copiar un archivo o directorio remoto.
`copy`	Alias de `cp`
`rename <roldpath> <rnewpath>`	Renombra un archivo o directorio remoto.
`rm <rpath>`	Borra un archivo remoto.
`chgrp <grp> <rpath>`	Cambiar el grupo a un archivo o directorio remoto
`chgrp <own> <rpath>`	Cambiar el propietario a un archivo o directorio remoto
`chmod <mode> <rpath>`	Cambiar las propiedades a un archivo o directorio remoto
`df <-h> <rpath>`	Muestra la información de uso del archivo o directorio remoto. Si activamos `-h` lo muestra en formato human- readable.
`cd <rpath>`	Cambia de directorio remoto.
`mkdir <rpath>`	Crea un directorio remoto.
`rmdir <rpath>`	Borra un directorio remoto.
`pwd`	Muestra el directorio remoto activo.

Comando	Descripción
Local	
`!<cmd>`	Ejecuta un comando del shell local o abre una shell interactiva.
`lcd <lpath>`	Cambia de directorio local.
`lls <lpath>`	Lista archivos locales.
`lmkdir <lpath>`	Crea un directorio local.
`lpwd`	Muestra directorio local activo.
Otros	
`help`	Muestra ayuda
`?`	Alias help

> En general todos los comandos admiten el uso de comodines en el nombre del archivo y/o directorio.

Conexión SFTP Interactiva

Conectar con sftp.ejemplo.com mediante un usuario alumno y contraseña p4bl1c0.

```
sftp alumno@sftp.ejemplo.com
#al entrar solicitara el password del usuario
alumno@sftp.ejemplo.com password: p4bl1c0
sftp>
```

Conexión SFTP Interactiva con clave privada

Conectar con sftp.ejemplo.com mediante un usuario alumno y su clave privada.

```
#Primero debemos generar la clave publica/privada en el cliente
#-q No pide confirmacion
#-N password ("" sin password)
#-t tipo de clave
#-Z cipher algoritmo de cifrado
#-f fichero para guardar la clave
ssh-keygen -q -N "" -t rsa -Z aes256-cbc -f ~/.ssh/rsa_alumno

#Publicamos la clave publica en el servidor
ssh-copy-id -i ./.ssh/rsa_alumno.pub alumno@sftp.ejemplo.com
#Al publicar la clave publica nos solicita la clave del usuario
alumno@sftp.ejemplo.com password: p4bl1c0

#Conectamos al servidor con la clave privada
sftp -i ~/.ssh/rsa_alumno alumno@ftp.ejemplo.com
```

Conexión SFTP No Interactiva

La conexion no interactiva hace referencia a que generaremos un fichero de ordenes sftp que utilizaremos para la conexión. Para realizar una conexión no interactiva es obligatorio el uso de la clave privada. Así usando la clave del ejemplo anterior.

```
#crear el archivo sftp_in.txt
lcd ~/local
cd  /alumno/remoto
get *.txt
put informe.pdf
bye
#Conectar al servidor con clave privada y archivo de ordenes
sftp -b sftp_in.txt -i ~/.ssh/rsa_alumno alumno@ftp.ejemplo.com
```

LFTP

El cliente `lftp` (man) es un cliente FTP avanzado para Linux. Entre sus características tenemos:

- Sintaxis de comandos similar a la de un shell.
- Compatible con FTP, FTPS y SFTP, HTTP, HTTPs
- Utiliza el formato URL. Ejemplo ftp://ftp.ejemplo.com
- Soporta scripting.
- Implementa reconexión automática ante errores.
- Ejecución de comandos de cliente dentro de la sesión FTP.
- Definición variables de entorno con `set`
- Ejecución de comandos en paralelo y segundo plano con `&`.
- Agrupación de comandos con `()`
- Ejecución de comandos concatenada con `&&` y `||`.
- Descarga, subida y sincronización de un arbol de directorios completo con `mirror`.
- Fichero de configuración en `/etc/lftp.conf` en el que se pueden configurar los `alias` o `set` de variables de inicio.
- Ejecución por defecto en modo pasivo.

El **cliente LFTP** sigue la siguiente sintáxis

```
lftp [-d] [-e cmd] [-p port] [-u user[,pass]] [site]
lftp -f script_file
lftp -c commands
lftp --version
lftp --help

-d   Activa el modo depuración
-e   Comando o comandos a ejecutar en el inicio de conexión.
     Se trata de una cadena de comandos separados por ;
     Ejemplo  "ls; pwd;"
-p   Puerto para conectar con servidor.
     Si no se indica se usan los puertos por defecto.
-u   usuario y password para la sesión
site url en la que iniciar sesión.
-f   fichero con comandos lftp.
-c   Similar a -e pero al terminar cierra la sesión.
--version indica la versión de lftp.
--help    ayuda.
```

En un shell de `lftp` podemos ejecutar la mayor parte de los comandos del `ftp` tradicional de linux, pero además incluye **comandos avanzados**:

Comando	Descripción
Sesion	
`alias <name> <value>`	Define un alias. Ejemplo: `alias dir ls -lF` `alias dir` (borra definición)
`set <var> <value>`	Define una variable. Opciones: `-a` Muestra todas las variables definidas `-d` Muestra las variables por defecto definidas en `/etc/lftp.conf`. Ejemplo: `set ftp:passive-mode false` Cambia modo de transferencia ftp a activo.
`local <cmd>`	Ejecuta un comando en la sesion local (`file://`). Ejemplo: `local mkdir -p /temp` Crea un directorio en el sistema local de archivos.
`exit`	Este comando cambia su comportamiento con respecto al exit de FTP. Exit termina la sesión lftp o bien se pone en background si tiene tareas todavía por ejecutar. Se puede modificar el comportamiento con las siguientes opciones: `bg` Fuerza a salir al modo background. `top` Terminar el shell interno principal de lftp. `parent` Termina el shell interno padre de lftp del que invocó exit `kill` Mata todas las tareas pendientes antes de salir
Tareas	
`&`	Ejecutar una tarea en segundo plano. Ejecución paralela. Ejemplo `get indice.txt &`
`jobs`	Lista las tareas en ejecución.
`kill <all>| <njob>`	Elimina todas las tareas o una tarea en concreto
`wait <all>| <njob>`	Espera a que terminan todas las tareas o una tarea en concreto
`fg`	Alias de wait.
`at <njob> <command>`	Espera a que termine una tarea antes de ejecutar `command`
`queue <command>`	Introduce el comando en una cola de comandos que se ejecutaran secuencialmente
`queue start`	Comienza la ejecución de comandos en cola
`queue stop`	Para la ejecucion de comandos en cola. Los comandos que ya se hubieran lanzado para la ejecución continuan ejecutandose.

Comando	Descripción
queue	Muestra el estado de cola de tareas
queue --delete <element>	Vacia la cola. Si no se indica element se borra el ultimo elemento. El elemento puede ser un número de indice o bien una cadena wildcard, por ejemplo, *. Ejemplos: queue --delete borra ultimo elemento queue --delete 5 borra elemento 5 queue --delete "*" borra toda la cola.
Ficheros	
get <opts> <rfile>	Descarga un fichero. Tiene más opciones que el FTP tradicional. Opciones: -c continuar si ocurre error temporal. -E borrar el origen después de la transferencia. -e borrar el destino antes de la transferencia. -a usar ASCCI (BINARY por defecto) -P <n> usar n conexiones en paralelo. -o <lfile> nombre del fichero local -O <ldir> directorio local. Ejemplo: get -c -P 4 -O "/temp" manual.mpg Descarga manual.mpg al directorio /temp usando 4 conexiones de datos en paralelo y reanudando descarga si hay errores temporales
put <opts> <lfile>	Sube un fichero. Tiene más opciones que el FTP tradicional. Opciones: -c continuar si ocurre error temporal. -E borrar el origen después de la transferencia. -e borrar el destino antes de la transferencia. -a usar ASCCI (BINARY por defecto) -P <n> usar n conexiones en paralelo. -o <rfile> nombre del fichero remoto -O <ldir> directorio local. Ejemplo: put -c -P 4 -O "/temp" -o "/videos/manual.mpg" manual.mpg Sube manual.mpg del directorio local /temp al archivo manual.mpg del directorio remoto /videos usando 4 conexiones de datos en paralelo y reanudando subida si hay errores temporales
Multiple	
mget <opts> <rfiles>	Descarga multiple de archivos mediante wildcard (*,?,[]). Tiene más opciones que el ftp tradicional. Opciones: -c continuar si ocurre error temporal. -d crear directorios sino existen -E borrar el origen después de la transferencia. -e borrar el destino antes de la transferencia. -a usar ASCCI (BINARY por defecto) -P <n> usar n conexiones en paralelo. -O <ldir> directorio local. Ejemplo: mget -c -d -P 4 -O "/temp" /videos/*. Descarga todos los archivos del directorio remoto

Comando	Descripción
	/videos/ (pero no sus subdirectorios), creará el directorio local /temp/videos, usará 4 conexiones simultáneas y reanudará las descargas en caso de interrupciones temporales.
`mput <opts> <lfiles>`	Sube multiples archivos mediante wildcard (*,?,[]). Tiene más opciones que el ftp tradicional. Opciones: `-c` continuar si ocurre error temporal. `-d` crear directorios sino existen `-E` borrar el origen después de la transferencia. `-e` borrar el destino antes de la transferencia. `-a` usa ASCCI (BINARY por defecto) `-P <n>` usar n conexiones en paralelo. `-O <ldir>` directorio local. Ejemplo: `mput -c -P 4 -O "/temp" videos/*`. Sube todos los archivos del directorio local /temp/videos/ (pero no sus subdirectorios), usará 4 conexiones simultáneas y reanudará la subida en caso de interrupciones temporales.
`mirror <opts> <source> <target>`	Realiza una copia en espejo entre `source` y `target`. Por defecto el origen es remoto y el destino local. La opción `-R` cambia el sentido. Puedes consultar las numerosas opciones de mirror en el manual de lftp. Ejemplos: `mirror -R -c -P 4 /docs/tutorials/ /man/tutorials/` Realiza una copia en espejo recursiva del directorio local /docs/tutorials al directorio remoto /man/tutorials. `mirror -c -P 4 /temp/videos/ /download/videos/` Realiza una copia en espejo recursiva del directorio remoto /temp/videos al directorio local /download/videos.

Conexión LFTP Mirror

Conectar con ftp.ejemplo.com mediante un usuario anonimo. Hacer un mirror de la carpeta remota /ser/03 a la carpeta local /ser/unidad03_ftp. Usar reconexión y proceso en paralelo de 5 conexiones y modo activo.

```
# Contenido de lftp_in.txt.
  # activar modo activo
    set ftp:passive-mode false
  # Parametros de rendimiento
    set net:timeout 20
    set net:max-retries 50
    set net:reconnect-interval-base 5
    set net:reconnect-interval-max 60
  # Conexion con servidor
    open -u anonymous,anonymous@ ftp://ftp.ejemplo.com
  # Crear directorio local
    local mkdir -p /ser/unidad03_ftp
  # mirroring
    mirror -c -P 5 /ser/03 /ser/unidad03_ftp
  # desconexion
    bye
```

```
# Ejecutar cliente ftp y redirigir la salida a un archivo.
lftp -f lftp_in.txt >lftp_out.txt 2>&1
```

Conexión LFTP Queue

Conectar con ftp.ejemplo.com mediante un usuario anonimo. Descargar mediante una cola varios archivos, esperar a que termine, y luego subier en cola varios archivos al servidor. Forzar el modo pasivo.

```
  # Contenido de lftp_in.txt.
# activar modo pasivo
  set ftp:passive-mode true
# Parametros de rendimiento
  set net:timeout 20
  set net:max-retries 50
  set net:reconnect-interval-base 5
  set net:reconnect-interval-max 60
# No iniciar colas automaticamente
  set queue:autostart off
# Conexion con servidor
  open -u anonymous,anonymous@ ftp://ftp.ejemplo.com
# descargar archivos
  queue local mkdir -p /ser/unidad03_ftp
  queue mget -c -P 2 -O /ser/unidad03_ftp /ser/03/*.pdf
  queue mget -c -P 2 -O /ser/unidad03_ftp /ser/03/*.csv
  queue get -c -O /ser/unidad03_ftp /ser/03/README.txt
  queue start
# esperar la terminacion
  wait
# crear cola para subir archivos
  queue mkdir -p /ser/03/upload
  queue cd /ser/03/upload
  queue mput -c -P 2 /ser/unidad03_ftp/upload/actividad*.*
  queue start
# esperar la terminacion
  wait
# finalizar sesion
  bye
```

```
  # Ejecutar cliente ftp y redirigir la salida a un archivo.
lftp -f lftp_in.txt >lftp_out.txt 2>&1
```

Conexión LFTP SFTP

Conectar con un servidor SFTP de nombre sftp.ejemplo.com mediante un usuario alumno y su clave privada generada anteriormente. Descargar mediante una cola varios archivos, esperar a que termine, y luego subier en cola varios archivos al servidor.

```
# Contenido de lftp_in.txt.
  # Parametro de conexion con clave privada
  set sftp:connect-program "ssh -i ~/.ssh/rsa_alumno"
  # Parametros de rendimiento
  set net:timeout 20
  set net:max-retries 50
  set net:reconnect-interval-base 5
  set net:reconnect-interval-max 60
  # No iniciar colas automaticamente
  set queue:autostart off
  # Conexion con servidor se utiliza la clave privada
```

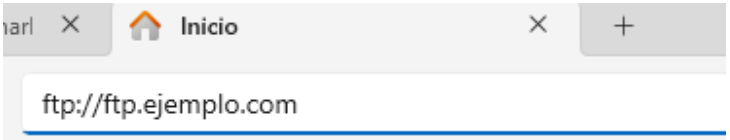
```
  open -u alumno sftp://sftp.ejemplo.com
# descargar archivos
  queue local mkdir -p /ser/unidad03_ftp
  queue mget -c -P 2 -O /ser/unidad03_ftp /ser/03/*.pdf
  queue mget -c -P 2 -O /ser/unidad03_ftp /ser/03/*.csv
  queue get -c -O /ser/unidad03_ftp /ser/03/README.txt
  queue start
# esperar la terminacion
  wait
# crear cola para subir archivos
  queue mkdir -p /ser/03/upload
  queue cd /ser/03/upload
  queue mput -c -P 2 /ser/unidad03_ftp/upload/actividad*.*
  queue start
# esperar la terminacion
  wait
# finalizar sesion
  bye
```

```
# Ejecutar cliente ftp y redirigir la salida a un archivo.
lftp -f lftp_in.txt >lftp_out.txt 2>&1
```

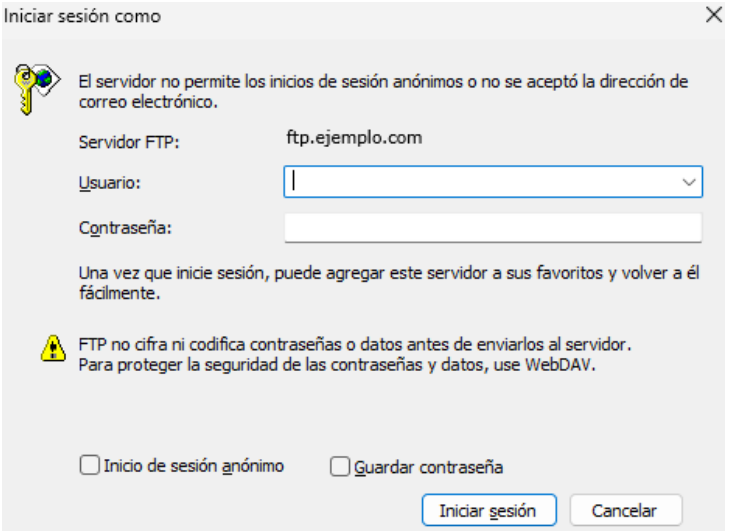
CLIENTES GRÁFICOS

EXPLORADORES DE ARCHIVOS

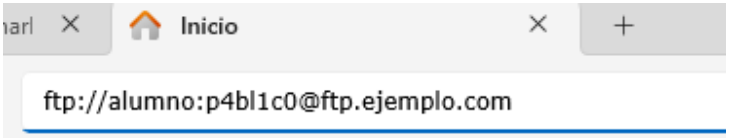
Los exploradores de archivos de los sistemas Windows y Linux son también **clientes ftp básicos**. Basta con indicar en la barra de direcciones del explorador la URL del servidor FTP, por ejemplo, ftp://ftp.ejemplo.com.



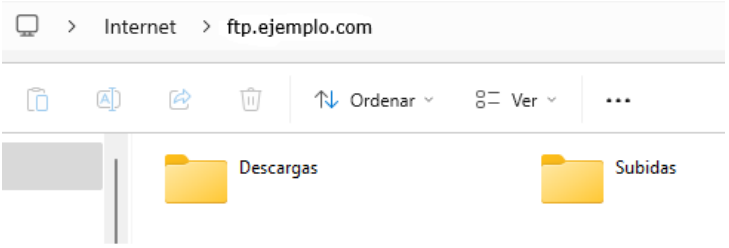
Posteriormente nos preguntará el usuario y contraseña para conectar con el servidor.



También podemos indicar directamente en la url del servidor el usuario y contraseña. Por ejemplo, si queremos acceder al servidor ftp.ejemplo.com mediante el usuario alumno y contraseña p4bl1c0 usamos la url ftp://alumno:p4bl1c0@ftp.ejemplo.com



Una vez accedido al servidor, las acciones de descargar archivos, subir archivos, crear carpetas,... se integran en las acciones básicas del explorador como son copiar, pegar, cortar,...



OTROS CLIENTES

Como hemos comentado existen otros clientes gráficos más avanzados que permiten por ejemplo la gestión individualizada de sitios FTP.
Entre los más utilizados tenemos:

- WinSCP
- Filezilla
- CuteFTP